

ADR 001 - Adoção da arquitetura de Microservices

Data: 11/08/2026 (Entrega) | **Status:** Aceito

Contexto

O domínio do sistema separa naturalmente duas responsabilidades com ciclos de mudança distintos: o fluxo de carrinho/pedido (checkout) e o processamento financeiro (pagamento), este último sujeito a regras de integração com gateways externos e a requisitos de segurança mais rígidos (PCI, idempotência de cobrança). A equipe é pequena e distribuída, precisando dividir o trabalho em fatias que possam ser desenvolvidas, testadas e entregues sem que uma pessoa bloqueie a outra. Um protótipo inicial em monolito único mostrou que qualquer alteração no módulo de pagamento exigia reimplantar todo o sistema, incluindo o catálogo/checkout, mesmo quando a mudança não afetava essa parte.

Decisão

Adotar arquitetura de Microservices, com o ms-checkout responsável pelo domínio de carrinho/pedidos e o ms-pagamento responsável pelo domínio financeiro, cada um com deploy e ciclo de release independentes. Consideramos um monolito modularizado (módulos internos separados por pacote, mesmo deploy), mas foi descartado porque não resolve o problema de acoplamento de deploy identificado no protótipo (uma falha ou mudança em pagamento continuaria arrastando o checkout). Consideramos também manter tudo em camadas dentro de um único serviço, descartado pelo mesmo motivo, além de dificultar a divisão de trabalho e a rastreabilidade de contribuição individual (commits por serviço).

Consequências

Positivo:

- Deploy independente: mudanças ms-pagamento não exigem reimplantar o ms-checkout.
- Escalabilidade independente (ex.: escalar só o ms-pagamento em picos de checkout).
- Facilita divisão de trabalho e evidência de autoria por serviço.

Negativo:

- Complexidade operacional maior: dois deploys, dois pipelines, necessidade de observabilidade distribuída (logs/tracing correlacionados).
- Introduz o problema de consistência entre serviços (pedido criado em um serviço, pagamento confirmado em outro), que exige uma estratégia de comunicação.
- Sem testes de contrato entre as APIs, uma mudança em um serviço pode quebrar o outro silenciosamente.